

THE UNIVERSITY OF CHICAGO

MASTERS IN COMPUTATIONAL SOCIAL SCIENCE
THE UNIVERSITY OF CHICAGO

MACS 30111

Dictionaries

Misc

- *Today:*
  - *Quiz:*
- *Midterm exam: 10/29 IN CLASS*

Topics:

- Introduction to Python dictionaries
- Useful dictionary operations
- Constructing dictionaries
- Iterating over dictionaries
- Accumulation
- Data structure and time complexity
- Sets

Quiz

What is a dictionary?

- Data structure
- Keys and values
- Can iterate through
- Can nest (!!)

Question: What is wrong, if anything?

```
def decr_and_remove(d):
    """
    Given a dictionary that maps keys to positive integers,
    decrement the values and remove any key-value pair in which
    the decremented value becomes zero

    Modifies the dictionary in-place.

    Args:
      d (Dict[str, int]): maps keys to positive integers

    Returns: None
    """
    for key, value in d.items():
        d[key] = value - 1
        if d[key] == 0:
            del d[key]
```

Caution

- Do not add/remove keys as you iterate over a dictionary

```
# INCORRECT!
for cand_id, cand_tot in cand_to_total.items():
    if cand_tot < 50000:
        del cand_to_total[cand_id]
```

Common pitfall: 2.2.1

Question: Differences? Which is better?

```
def total_contributions_by_campaign(contributions):
    """
    Compute total contributions by campaign

    Args:
      contributions (List[Dict[str, (str | float)]]): one dictionary per
        contribution

    Returns (Dict[str, float]): a dictionary that maps a campaign
name to the
        total contributions to the campaign.
    """
    rv = {}
    for contribution in contributions:
        campaign = contribution["campaign"]
        rv[campaign] = rv.get(campaign, 0) +
contribution["amount"]
    return rv
```

```
def total_contributions_by_campaign(contributions):
    """
    Compute total contributions by campaign

    Args:
      contributions (List[Dict[str, (str | float)]]): one dictionary per
        contribution

    Returns (Dict[str, float]): a dictionary that maps a campaign
name to the
        total contributions to the campaign.
    """
    rv = {}
    for contribution in contributions:
        campaign = contribution["campaign"]
        if campaign not in rv:
            rv[campaign] = 0
        rv[campaign] += contribution["amount"]
    return rv
```

Values can have any type

- Values can have any type, including dictionaries, list of dictionaries, etc.

Sample candidate:

First Name: Sam
Last Name: Seaborn
Party: DEM
City: Laguna Beach
State: CA
ZIP Code: 92651
Candidate ID: C00002600
District: 47

```
{ "First Name":"Sam",
  "Last Name": "Seaborn",
  "Party": "DEM",
  "City": "Laguna Beach",
  "State": "CA",
  "ZIP Code": "92651",
  "Candidate ID":"C00002600",
  "District": 47
 }
```

Nested dictionaries

```
{"Name":
     {"First Name": "Sam",
      "Last Name": "Seaborn"},
      "Party": "DEM",
      "Office Location":
           {"City": "Laguna Beach",
            "State": "CA",
            "ZIP Code": "92651"},
      "Candidate ID":"C00002600",
      "District": 47
 }
```

Application

In class exercise folder

Run the tests: one line per test case

In the folder with both files, run

```
pytest -v test_M3a_dict_debug.py
```

The top of the output: every test case, PASSED or FAILED

```
test_M3a_dict_debug.py::test_contributions_by_campaign[empty] PASSED              [  9%]
test_M3a_dict_debug.py::test_contributions_by_campaign[one_contribution] FAILED   [ 18%]
test_M3a_dict_debug.py::test_contributions_by_campaign[five_contributions] FAILED [ 27%]
test_M3a_dict_debug.py::test_total_by_campaign_by_zip[empty] PASSED               [ 36%]
test_M3a_dict_debug.py::test_total_by_campaign_by_zip[one_contribution] PASSED    [ 45%]
test_M3a_dict_debug.py::test_total_by_campaign_by_zip[same_campaign_same_zip] FAILED [ 54%]
test_M3a_dict_debug.py::test_total_by_campaign_by_zip[five_contributions] FAILED  [ 63%]
test_M3a_dict_debug.py::test_drop_small_campaigns[empty] PASSED                   [ 72%]
test_M3a_dict_debug.py::test_drop_small_campaigns[keep_all] PASSED                [ 81%]
test_M3a_dict_debug.py::test_drop_small_campaigns[drop_one] FAILED                [ 90%]
test_M3a_dict_debug.py::test_drop_small_campaigns[drop_all] FAILED                [100%]
```

Ask: Every [empty] case passes, even with the bugs. Why test with 1 and 5 contributions too?

11 test cases = 3 test functions × their inputs. Next: scroll to the bottom for the summary.

Troubleshooting: start at the bottom

Run pytest test_M3a_dict_debug.py. The summary at the bottom lists every test case that failed.

```
=========== short test summary info ===========
FAILED test_M3a_dict_debug.py::test_contributions_by_campaign[one_contribution]
FAILED test_M3a_dict_debug.py::test_contributions_by_campaign[five_contributions]
FAILED test_M3a_dict_debug.py::test_total_by_campaign_by_zip[same_campaign_same_zip]
FAILED test_M3a_dict_debug.py::test_total_by_campaign_by_zip[five_contributions]
FAILED test_M3a_dict_debug.py::test_drop_small_campaigns[drop_one] - RuntimeError
FAILED test_M3a_dict_debug.py::test_drop_small_campaigns[drop_all] - RuntimeError
================ 6 failed, 5 passed in 0.05s ================
```

Each FAILED line is one broken test case. Start with the first one.

Troubleshooting: read one FAILED line

```
test_M3a_dict_debug.py::test_total_by_campaign_by_zip[same_campaign_same_zip]
```

test_M3a_dict_debug.py

The test file

Where the tests live. Open this next.

test_total_by_campaign_by_zip

The test function

test_ + the name of the function being checked

[same_campaign_same_zip]

The test case

One set of inputs and the expected output

Next: find that function and that case in the test file.

Test file: test_M3a_dict_debug.py

```
import pytest
from M3a_dict_debug import (
    contributions_by_campaign,
    total_by_campaign_by_zip,
    drop_small_campaigns,
)

KANG = "Kang for President 2016"

# small data you can check by eye
C1 = {"zip_code": "60637", ...
     "campaign": KANG, "amount": 27.50}
C4 = {"zip_code": "60637", ...
     "campaign": KANG, "amount": 50.00}
```

1. Imports the three functions you are fixing

2. Builds a tiny dataset, so you can work out the right answer yourself

3. Below this: one test function per function in M3a_dict_debug.py

Test function: many test cases

```
@pytest.mark.parametrize(
   "contributions, expected",
   [
      ([], {}),
      ([C1], {KANG: {"60637": 27.5}}),
      ([C1, C4], {KANG: {"60637": 77.5}}),  <-
      (CONTRIBUTIONS, {...}),
   ],
   ids=["empty", "one_contribution",
       "same_campaign_same_zip",
       "five_contributions"],
)
def test_total_by_campaign_by_zip(contributions,
                                  expected):
   assert total_by_campaign_by_zip(contributions) \
      == expected
```

Each row is one test case:

(inputs, expected output)

ids name the cases.

They show up in [brackets] in the output.

assert compares

your output to the expected output.

Test values: find the trouble case

The failing case: same_campaign_same_zip

```
inputs:   [C1, C4]   # both KANG, both ZIP 60637: 27.50 + 50.00
expected: {KANG: {"60637": 77.5}}
```

What pytest says

```
got:      {'Kang for President 2016': {'60637': 50.0}}
expected: {'Kang for President 2016': {'60637': 77.5}}
```

Ask: What did we get? What did we expect? Where did the 27.50 go?

Rerun just this case

```
pytest -v test_M3a_dict_debug.py -k same_campaign_same_zip
```

Failure type 1: the wrong answer

From the FAILURES section: test_contributions_by_campaign[one_contribution]

```
    def test_contributions_by_campaign(contributions, expected):
>       assert contributions_by_campaign(contributions) == expected
E       AssertionError: assert {} == {'Kang for Pr... 'Doe', ...}]}
E
E         Right contains 1 more item:
E         {'Kang for President 2016': [{'amount': 27.5, ...}]}

test_M3a_dict_debug.py:50: AssertionError
```

> marks the line

The assert that failed, in the test file (line 50).

Left of ==

What your function returned: an empty dict, {}.

Right of ==

What the test expected. "Right contains 1 more item" = a key you never added.

Ask: The loop appended C1 to a list. Which list, and where did it go?

Failure type 2: the code crashed

From the FAILURES section: test_drop_small_campaigns[drop_one]

```
test_M3a_dict_debug.py:94:
_ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
totals = {'Kang for President 2016': 87.5, 'Kodos for President 2016': 100.0}
min_total = 50.0

    def drop_small_campaigns(totals, min_total):
        ...
>       for campaign, total in totals.items():
E       RuntimeError: dictionary changed size during iteration

M3a_dict_debug.py:88: RuntimeError
```

No got vs. expected

The function never returned. There is no answer to compare.

Follow the trail

The test (line 94) called your code. The last stop is the bug: M3a_dict_debug.py:88.

Read the values

totals has 2 campaigns, not 3. Krusty was deleted before the crash.

Ask: Where have we seen this error before today?

Get the class code: fork the repo

Once per quarter, on github.com

1. **Open the course repo**
   github.com/jmclip/macs30111

2. **Click Fork (top right), then Create fork**
   You now have your own copy: github.com/YOUR-USERNAME/macs30111

3. **Copy YOUR fork's URL**
   Green Code button → HTTPS. Check that it says your username, not jmclip.

Then, in the terminal

```
# 4. download your fork
git clone https://github.com/
    YOUR-USERNAME/macs30111.git

# 5. go to today's files
cd macs30111/in_class_exercises
ls
```

You should see **M3a_dict_debug.py** and **test_M3a_dict_debug.py**.

Ask: Why fork instead of cloning jmclip/macs30111 directly? (Hint: who can push to it?)

New files posted? Update your fork

Your fork doesn't update itself. Before class, pull in what's new in jmclip/macs30111.

Option A: GitHub website, then terminal

```
# on github.com/YOUR-USERNAME/macs30111:
#   Sync fork → Update branch

# then bring it to your computer:
cd macs30111
git pull
```

Option B: all in the terminal

```
# once: remember the course repo
git remote add upstream \
  https://github.com/jmclip/macs30111.git

# every update:
git pull upstream main
git push
```

Commit your work first

git add + git commit before you update, so nothing gets overwritten.

Merge conflict?

You and the course repo changed the same lines. Don't panic: ask us.

Check it worked

ls should show the new files, e.g. M3a_dict_debug.py.

Function 1: contributions_by_campaign

In M3a_dict_debug.py (docstring trimmed)

```
def contributions_by_campaign(contributions):
    rv = {}
    for contribution in contributions:
        campaign = contribution["campaign"]
        rv.get(campaign, []).append(contribution)
    return rv
```

1

Should return

A dict mapping each campaign to the list of its contributions, in their original order.

2

Its tests

empty **PASSED**
one_contribution **FAILED**
five_contributions **FAILED**

3

Ask

pytest says this returned {}. Which line was supposed to add a key to rv?

Function 2: total_by_campaign_by_zip

In M3a_dict_debug.py (docstring trimmed)

```
def total_by_campaign_by_zip(contributions):
    by_campaign_by_zip = {}
    for contribution in contributions:
        campaign = contribution["campaign"]
        zip_code = contribution["zip_code"]
        by_campaign_by_zip[campaign] = {}
        inner = by_campaign_by_zip[campaign]
        inner[zip_code] = inner.get(zip_code, 0) + contribution["amount"]
    return by_campaign_by_zip
```

1

Should return

A nested dict: each campaign maps to a dict of ZIP Code → total amount from that ZIP.

2

Its tests

empty **PASSED**
one_contribution **PASSED**
same_campaign_same_zip **FAILED**
five_contributions **FAILED**

3

Ask

One contribution works, but two to the same campaign don't. What happens on the second time through the loop?

Function 3: drop_small_campaigns

In M3a_dict_debug.py (docstring trimmed)

```
def drop_small_campaigns(totals, min_total):
    for campaign, total in totals.items():
        if total < min_total:
            del totals[campaign]
    return totals
```

1

Should return

A NEW dict with only the campaigns that raised at least min_total. The input must not change.

2

Its tests

empty **PASSED**
keep_all **PASSED**
drop_one **FAILED** RuntimeError
drop_all **FAILED** RuntimeError

3

Ask

Both failures are crashes, not wrong answers. Why do empty and keep_all get through?

Debugging lab

Files: M3a_dict_debug.py, test_M3a_dict_debug.py

1. Run the tests with your partner
2. Read each failure: which test? which input? expected vs. actual?
3. Fix each bug with as few changes as possible
4. Add a # FIX: comment above each fix saying what was wrong
5. All tests pass? You're done

Commands

```
# run everything
pytest -v test_M3a_dict_debug.py

# run one function's tests
pytest -v test_M3a_dict_debug.py \
   -k drop_small
```

Three functions, one bug each.

Done = every test passes.

Lab check-in: passing isn't just the right answer

A tempting fix: collect the keys, then delete

```
to_drop = [c for c, t in totals.items() if t < min_total]
for campaign in to_drop:
    del totals[campaign]
return totals
```

pytest -v test_M3a_dict_debug.py -k drop_small

```
    original = copy.deepcopy(totals)
    assert drop_small_campaigns(totals, min_total) == expected
>   assert totals == original, "...modified its input!"
E   AssertionError: drop_small_campaigns modified its input!
E     Right contains 1 more item:
E     {'Krusty for President 2016': 5.0}

test_M3a_dict_debug.py:95: AssertionError
====== 2 failed, 2 passed, 7 deselected in 0.03s ======
```

Line 94 passed

The function returned the right dict. No crash.

Line 95 failed

The test also checks that your input is unchanged.

-k drop_small

Only 4 of 11 cases ran. The other 7 were deselected.

Ask: The docstring promises a NEW dictionary. How do you build one?

Done: every line says PASSED

pytest -v test_M3a_dict_debug.py, after all three fixes

```
test_M3a_dict_debug.py::test_contributions_by_campaign[empty] PASSED            [ 9%]
test_M3a_dict_debug.py::test_contributions_by_campaign[one_contribution] PASSED      [ 18%]
test_M3a_dict_debug.py::test_contributions_by_campaign[five_contributions] PASSED    [ 27%]
test_M3a_dict_debug.py::test_total_by_campaign_by_zip[empty] PASSED             [ 36%]
test_M3a_dict_debug.py::test_total_by_campaign_by_zip[one_contribution] PASSED       [ 45%]
test_M3a_dict_debug.py::test_total_by_campaign_by_zip[same_campaign_same_zip] PASSED [ 54%]
test_M3a_dict_debug.py::test_total_by_campaign_by_zip[five_contributions] PASSED     [ 63%]
test_M3a_dict_debug.py::test_drop_small_campaigns[empty] PASSED                 [ 72%]
test_M3a_dict_debug.py::test_drop_small_campaigns[keep_all] PASSED              [ 81%]
test_M3a_dict_debug.py::test_drop_small_campaigns[drop_one] PASSED              [ 90%]
test_M3a_dict_debug.py::test_drop_small_campaigns[drop_all] PASSED              [100%]

============================== 11 passed in 0.01s ==============================
```

1. Add # FIX: comments

One above each fix: what was wrong, and why.

2. Rerun once more

Comments can't break code, but check anyway.

3. Compare with a neighbor

Did you fix each bug the same way?

TT3: all fun, all the time

Pull up all materials

TT3 Description: https://classes.ssd.uchicago.edu/macss/macs30121/modules/tt/tt3.html
TT3 Assignment: https://classroom50.org/MACS-30111-23/macs-30111-fall-26/assignments/tt3-class/accept

Before Thursday

- Finish the debugging lab
- TT3: work through Tasks 1–7 in cfpb.py
  - Task 7 builds on your Tasks 3 and 6
- Thursday: Task 8 together. Bring your code!

Summary

- Dictionaries are very useful for accumulating values associated with keys.

- The in operator, the not in operator, and the get method all allow us to handle previously seen and previously unseen keys cleanly.

Day 2: TT3, continued

TT3: all fun, all the time

Pull up all materials

TT3 Description: https://classes.ssd.uchicago.edu/macss/macs30121/modules/tt/tt3.html
TT3 Assignment: https://classroom50.org/MACS-30111-23/macs-30111-fall-26/assignments/tt3-class/accept

Misc

- Today:
  - We will use information from TT3: https://classes.ssd.uchicago.edu/macss/macs30121/modules/tt/tt3.html
  - Materials: https://classroom50.org/MACS-30111-23/macs-30111-fall-26/assignments/tt3-class/accept

Topics:

- Introduction to Python dictionaries
- Useful dictionary operations
- Constructing dictionaries
- Iterating over dictionaries
- Accumulation
- **Sets**
- **Dictionaries as objects**
- **Data structure and time complexity**

Sets

- Making a set: set([list])
- Only count things once
- add() to add something
- update([list]) to add multiple
- remove() or discard() to remove an element
  - If you choose something that isn't present, only 'remove' will let you know with an error message
- union() or `|' will bring together sets
- intersection() or `&' will return common elements
- difference() returns only elements unique to first set

https://www.pythoncheatsheet.org/cheatsheet/sets

Sets v Lists

- Set tasks:
  - Make a set, s1, that contains the following elements: 1,2,3,3,4,5,4,3,4
    - How many elements will it contain?
  - Add 3 to your set
  - Add 5,6,7 to your set
  - Compare s1 with a second set, s2, that contains 3, 6,9.
  - What will s1.difference(s2) produce?

Building dictionaries with zip()

zip() pairs up two lists; dict() turns the pairs into key/value entries.

```
header = ["Company", "State", "Product"]
row    = ["Acme Anvils", "IL", "Mortgage"]

complaint = dict(zip(header, row))
# {'Company': 'Acme Anvils', 'State': 'IL', 'Product': 'Mortgage'}

{k: v for k, v in zip(header, row)}   # same result
```

Predict:

```
dict(zip(["IL", "CA"], [12, 30, 7]))
```

Exercise: TT3

- Load your files:

```
import json
CFPB_16 = json.load(open("cfpb16_1000.json"))
```

(alternative: import cfpb)

- What is CFPB_16? What structure does it have?

Exercise: TT3

- Get the keys from cfpb.CFPB_16 or CFPB_16

Anatomy of a script

```
# Task 1
    def find_companies(complaints):

# Task 2
    def count_complaints_about(complaints, company_name):

# Task 3
    def count_by_state(complaints):

# Task 4
    def state_with_most_complaints(cnt_by_state):

# Task 5
    def count_by_company_by_state(complaints):

# Task 6
    def complaints_by_company(complaints):

# Task 7
    def count_by_company_by_state_2(complaints):
```

Prepping Task 1

- **Task 1:** In cfpb.py, write a function
  - `def find_companies(complaints):`

- that **takes a list of complaints and returns a list** (**or set** — see above) of the companies that received at least one complaint.
- Remember: we've included a variable called CFPB_16 that contains information from 1000 complaints in 2016. You will be using this variable when testing these functions, as shown below.

TT3: PARTY!

- Look over the TT3 code if you have not done so already
- Create a map of the functions.
- Draw lines to show where / how functions are called / used by others

Anatomy of a script

```
# Task 1
    def find_companies(complaints):

# Task 2
    def count_complaints_about(complaints, company_name):

# Task 3
    def count_by_state(complaints):

# Task 4
    def state_with_most_complaints(cnt_by_state):

# Task 5
    def count_by_company_by_state(complaints):

# Task 6
    def complaints_by_company(complaints):

# Task 7
    def count_by_company_by_state_2(complaints):
```

Anatomy of a script

```
# Task 1
    def find_companies(complaints):

# Task 2
    def count_complaints_about(complaints, company_name):

# Task 3
    def count_by_state(complaints):

# Task 4
    def state_with_most_complaints(cnt_by_state):

# Task 5
    def count_by_company_by_state(complaints):

# Task 6
    def complaints_by_company(complaints):

# Task 7
    def count_by_company_by_state_2(complaints):
```



TT3: task 8

TT3: comparing code

Task 8: With your team, compare the implementations of count_by_company_by_state and count_by_company_by_state_2, and discuss the advantages and disadvantages of each.

Some aspects to consider:

1. Which is easier to read and understand?
2. Which uses less code?
3. Which is easier to debug?
4. Which is faster?
    - As a proxy for speed, you can ask:
        - Which requires fewer passes through the data?
        - Which implementation would you choose?

TT3: function comparison

```
def count_by_company_by_state(complaints):
'''
Computes a dict of {company: {state: count, state: count}} for all states
and companies

Inputs:
complaints (list) A list of complaints, where each complaint is a
dictionary

Returns: (dict) with count per company per state
'''


by_company_by_state = {}
for complaint in complaints:
        company = complaint["Company"]
        state = complaint["State"]
                if company not in by_company_by_state:
                        by_company_by_state[company] = {}
                if state not in by_company_by_state[company]:
                        by_company_by_state[company][state] = 0
                        by_company_by_state[company][state] += 1


return by_company_by_state
```

TT3: function comparison

```
def count_by_company_by_state_2(complaints):
'''
Computes a dict of {company: {state: count, state: count}} for all states and companies

Inputs: complaints (list) A list of complaints, where each complaint is a dictionary

Returns: (dict) with count per company per state

This implementation involves composing complaints_by_company with count_by_state
'''

by_company = complaints_by_company(complaints)
by_company_by_state = {company: count_by_state(company_complaints) for company, company_complaints in
by_company.items()}
return by_company_by_state
```

Data structure and time complexity

- Your choice of data structure can have a considerable impact on code efficiency
- List: running time proportional to the size of the list $O(n)$ because we need to iterate over the list
- Dict: implemented using a hash table that is optimized to access key-value mappings very quickly, in constant time $O(1)$

	N = 5000	N = 20000	N = 43582
Lists	0.158 ms	0.572 ms	1.17 ms
Dictionaries	0.00471 ms	0.00465 ms	0.00498 ms

%timeit

Coding practice: 2.2.5

Big-O notation

- Complexity of Python operations: https://wiki.python.org/moin/TimeComplexity

- If n is the size of the input to a problem:
  - $O(n^2)$ means the running time is roughly proportional to n^2 (i.e., as n gets bigger, the running time grows quadratically).
  - $O(n)$ means the running time grows linearly.
  - $O(\log n)$ means the running time grows logarithmically.
  - $O(1)$ means the running time is constant.

Bad keys

```
1 d = {0.3: "found"}
```

```
1 d[0.1+0.1+0.1]
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-2-da5394697656> in <module>
----> 1 d[0.1+0.1+0.1]

KeyError: 0.30000000000000004
```

- Computers use binary system instead of a base 10 system
- 0.1 and 0.3 can't be precisely represented in a binary system
- The binary representation of 0.3 is not equal to that of 0.1+0.1+0.1

Dictionaries and data: who, what, where, when, why, how

- Use to work with data
- Often when you are working with a database
- PROBABLY NOT if you're trying to do data manipulation
- Think of it as a super structured table
- Can use to create a database (hello, R!)

Wrap-up exercise

PA2 (!!!)

- Review of initial assignment
- Importance of sketching
- DUE DATE 11/6 START AHEAD!!!

PA2: the “Technical Challenge”



Recipes contain helpful instructions like:

- Make cookies
- Make custard
- Bake

Image source

PA2

- Language model
- Based on cellular automata
- VERY OPEN ENDED
- For the model, specify:
  - a region,
  - the speakers in a region,
  - a speaker's neighborhood,
  - community centers in a region,
  - a speaker's engagement level,
  - language transmission rules,
  - a step in the simulation, and
  - the stopping conditions for the simulation.

PA2

- Suggested workflow:
- Sketch out what you will need
- “ Big rocks first”:
  - README files!!
    - Main file + test folder
  - Sketch (rough draft ON PAPER)
  - Functions: (what do you need to include to reduce errors)?
  - Docstrings
  - Text
- TEST AS YOU GO!!

Review: from the reading

Dictionaries

- What do we know?
- What are they like?
- What are they NOT like?
- When would we use them?

Review Lists:

- List creation and basic usage
- List iteration
- Adding, removing elements from a list
- List slicing
- Other operations
- Tuples
- Strings
- List Comprehensions
- Lists in Memory

```
lang = ["C", "C++", "Python", "Java"]
```

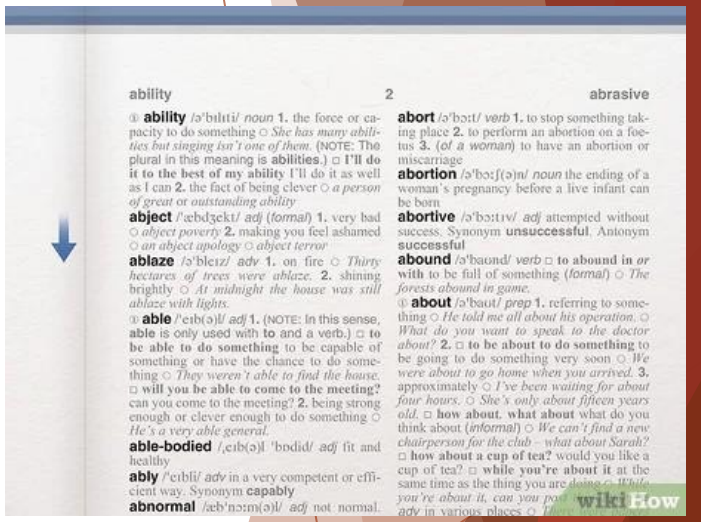
Representing candidates using a *dictionary*

- Map keys to values
- Each value is associated with a unique key rather than a position in a sequence

Sample candidate:

First Name: Sam
Last Name: Seaborn
Party: DEM
City: Laguna Beach
State: CA
ZIP Code: 92651
Candidate ID: C00002600
District: 47

```
{ "First Name":"Sam",
  "Last Name": "Seaborn",
  "Party": "DEM",
  "City": "Laguna Beach",
  "State": "CA",
  "ZIP Code": "92651",
  "Candidate ID":"C00002600",
  "District": 47
 }
```



Coding practice: 2.2

```
['Sam', 'Seaborn', 'DEM', 'Laguna Beach', 'CA', '92651', 'C00002600', 47]
```

Access information in a dictionary

```
d = {"first_name": "John",
     "last_name": "Doe",
     "zip_code": "60637",
     "campaign": "Kang for President 2016",
     "amount": 27.50}
```

- subscript notation
- *get* method
- *in* operator

```
d["zip_code"]
```

```
d.get("affiliation", "Unknown")
```

```
"first_name" in d
```

Coding practice: 2.2.1

Updating dictionaries

```
d = {"first_name": "John",
     "last_name": "Doe",
     "zip_code": "60637",
     "campaign": "Kang for President 2016",
     "amount": 27.50}
```

- Updating the value associated with a key
- Adding new key value pairs
- Remove an entry

```
d["zip_code"] = "94305"
```

```
d["affiliation"] = "Kodosican"
```

```
del d["affiliation"]
```

Coding practice: 2.2.1

Dictionaries

```
d = {"first_name": "John", "last_name": "Doe", "zip_code": "60637",
"campaign": "Kang for President 2016", "amount": 27.50}

d.get("first_name")
```

What is the difference?

d.get("affiliation") vs d.get("affiliation", "Unknown")

Iterating over dictionaries

- keys() yields a list-like object with the keys in a dictionary
- values( ) yields a list-like object with the values in a dictionary
- items( ) yields a list-like object with key/value tuples from a dictionary

Used in conjunction with for loops to iterate over dictionaries.

Coding practice: 2.2.1

Construct *dict*: dictionary literals

```
d = {"first_name": "John",
     "last_name": "Doe",
     "zip_code": "60637",
     "campaign": "Kang for President 2016",
     "amount": 27.50}
```

Coding practice: 2.2.2

Construct *dict*: empty dictionary + dictionary updates

- Start with an empty dictionary
- Add key/value pairs using the subscript notation

```
d = {}

d["first_name"] = "John"
d["last_name"] = "Doe"
d["zip_code"] = "60637"
d["campaign"] = "Kang for President 2016"
d["amount"] = 27.50
d["affiliation"] = "Kodosican”
d
```

TT3: How would we get the first element? How would we get the first element’s product?

Construct *dict*: dict constructor

- Use the dict constructor with a list of key-value pairs or another dictionary

```
d = {"first_name": "John", "last_name": "Doe",
"zip_code": "60637", "campaign": "Kang for President
2016", "amount": 27.50}

keys_and_data = [("first_name", "John"),
("last_name", "Doe"), ("zip_code", "60637"),
("campaign", "Kang for President 2016"), ("amount",
27.50)]

d_from_list = dict(keys_and_data)

d_from_dict = dict(d)
```

Construct *dict*: dictionary comprehensions

d = {key: value for key, value in keys_and_data}

```
<list name> = [ <transformation expression> for <variable name>
                                  in <list expression> ]
```

Coding practice: 2.2.2

Representing campaign data

Represent political candidates and contributions.

Sample candidate:	**Sample contribution:**
First Name: Sam	**Candidate ID: C00002600**
Last Name: Seaborn	Amount: $1000
Party: DEM	City: Silver Spring
City: Laguna Beach	State: MD
State: CA	ZIP Code: 20902
ZIP Code: 92651	Month: 11
Candidate ID: C00002600	Year: 2003
District: 47	

```
['Sam', 'Seaborn', 'DEM', 'Laguna Beach', 'CA', '92651', 'C00002600', 47]
['C00002600', '$1000', 'Silver Spring', 'MD', '20902', 11, 2003]
```

Disadvantages

Representing contributions

- Keys are often strings, but other immutable types (integers, booleans) can be used as well.
- Values can have any type, including different types for different values.

```
{ "Cand_ID":"C00002600",
  "Amount": 1000,
  "City": "Silver Spring",
  "State": "MD",
  "ZIP Code": "20902",
  "Month": 11,
  "Year": 2003 }
```

Sample contribution

Candidate ID: C00002600

Amount: $1000

City: Silver Spring

State: MD

ZIP Code: 20902

Month: 11

Year: 2003

Representing contributions

Sample contribution

Candidate ID: C00002600

Amount: $1000

City: Silver Spring

State: MD

ZIP Code: 20902

Month: 11

Year: 2003


```
{ "Cand_ID":"C00002600",
  "Amount": 1000,
  "City": "Silver Spring",
  "State": "MD",
  "ZIP Code": "20902",
  "Month": 11,
  "Year": 2003 }
```

Total contributions for all candidates

- Mapping from candidate IDs to total contributions received by the candidates
- Accumulate values based on keys


```
{ "Cand_ID":"C00002600",
  "Amount": 1000,
  "City": "Silver Spring",
  "State": "MD",
  "ZIP Code": "20902",
  "Month": 11,
  "Year": 2003 }
```


```
cand_to_total =
    {'C00002600': 433680,
     'C00012229': 469755,
     'C00013128': 398652,
     'C00017830': 561314,
     'C00019075': 538150,
         …
    }
```

Coding practice: 2.2.3

Dictionaries as objects

- Dictionaries are also commonly used to store "objects"
- E.g., keep track of programming assignments

```
pa1 = {"name": "Programming Assignment #1",
       "short_name": "pa1",
       "deadline": "2022/10/12",
       "num_submissions": 154}

pa2 = {"name": "Programming Assignment #2",
       "short_name": "pa2",
       "deadline": "2022/10/19",
       "num_submissions": 78}

pa3 = {"name": "Programming Assignment #3",
       "short_name": "pa3",
       "deadline": "2022/10/26",
       "num_submissions": 0}
```

```
1  pas = [pa1, pa2, pa3]
2
3  for pa in pas:
4      print(pa["name"], "is due on", pa["deadline"])
```

```
Programming Assignment #1 is due on 2021/10/15
Programming Assignment #2 is due on 2021/10/22
Programming Assignment #3 is due on 2021/10/29
```

Virtual environments

- Setting up and managing: conda vs venv
- Installs: pip vs conda